```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
```

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
from nltk.tokenize import word_tokenize,sent_tokenize
```

```
sent="I LOVE my India and it is the greatest country and that is a lie consider it "
print(word_tokenize(sent))
print(sent_tokenize(sent))
```

```
['I', 'LOVE', 'my', 'India', 'and', 'it', 'is', 'the', 'greatest', 'country', 'and', 'that', 'is', 'a', 'lie', 'consider', 'it']
['I LOVE my India and it is the greatest country and that is a lie consider it']
```

```
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words=set(stopwords.words("english"))
print(stop_words)
```

```
{'both', 'this', "he'll", 'under', "hadn't", 'just', 'ours', 'you', 'didn', 'aren', 'then', "mustn't", 'needn', 'his', 'them', '
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
token=word_tokenize(sent)
cleaned_token=[]
for w in token:
  if w not in stop_words:
    cleaned_token.append(w)
print("Tokenized Sentence:",token)
print("Filterd Sentence:",cleaned_token)
```

```
Tokenized Sentence: ['I', 'LOVE', 'my', 'India', 'and', 'it', 'is', 'the', 'greatest', 'country', 'and', 'that', 'is', 'a', 'lie
Filterd Sentence: ['I', 'LOVE', 'India', 'greatest', 'country', 'lie', 'consider']
```

```
w=[cleaned_token.lower() for cleaned_token in cleaned_token if cleaned_token.isalpha()]
print(w)
```

```
['i', 'love', 'india', 'greatest', 'country', 'lie', 'consider']
```

```
from nltk.stem import PorterStemmer
stemmer=PorterStemmer()
port_stemmer_output=[stemmer.stem(w) for w in w]
print(port_stemmer_output)
```

```
['i', 'love', 'india', 'greatest', 'countri', 'lie', 'consid']
```

```
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()
lemmatizer_output = [lemmatizer.lemmatize(w) for w in w]
print(lemmatizer_output)
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
['i', 'love', 'india', 'greatest', 'country', 'lie', 'consider']
```

```
from nltk.corpus.reader import tagged
nltk.download('averaged_perceptron_tagger_eng')
```

```
from nltk import pos_tag
token =word_tokenize(sent)
cleaned_token=[]
for w in token:
  if w not in stop_words:
    cleaned_token.append(w)
tagged=pos_tag(cleaned_token)
print(tagged)
```

```
[('I', 'PRP'), ('LOVE', 'VBP'), ('India', 'NNP'), ('greatest', 'JJS'), ('country', 'NN'), ('lie', 'NN'), ('consider', 'VB')]
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
documentA="Jupyter is the largest Planet"
documentB="Mars is the fourth planet from Sun"
```

```
bagofWordsA=documentA.split(' ')
bagofWordsB=documentB.split(' ')
```

```
uniquewords=set(bagofWordsA).union(set(bagofWordsB))
```

```
numofWordsA = dict.fromkeys(uniquewords, 0)
for word in bagofWordsA:
    numofWordsA[word] += 1
numofWordsB = dict.fromkeys(uniquewords, 0)
for word in bagofWordsB:
    numofWordsB[word] += 1
```

```
def computeTF(wordDict,bagofWords):
    tfDict = {}
    bagofWordscount=len(bagofWords)
    for w,count in wordDict.items():
      tfDict[w]=count /float(bagofWordscount)
    return tfDict
tfA = computeTF(numofWordsA, bagofWordsA)
tfB = computeTF(numofWordsB, bagofWordsB)
```

```
import math
def computeIDF(documents):
  N = len(documents)
  idfDict = dict.fromkeys(documents[0].keys(), 0)
  for document in documents:
    for word, val in document.items():
      if val > 0:
        idfDict[word] += 1
  for word, val in idfDict.items():
    idfDict[word] = math.log(N / float(val))
  return idfDict
idfs = computeIDF([numofWordsA, numofWordsB])
idfs
```

```
{'largest': 0.6931471805599453,
 'Planet': 0.6931471805599453,
 'planet': 0.6931471805599453,
 'is': 0.0,
 'Mars': 0.6931471805599453,
 'from': 0.6931471805599453,
 'Jupyter': 0.6931471805599453,
 'Sun': 0.6931471805599453,
 'the': 0.0,
 'fourth': 0.6931471805599453}
```

```
def computeTFIDF(tfBagOfWords, idfs):
  tfidf = {}
  for word, val in tfBagOfWords.items():
    tfidf[word] = val * idfs[word]
  return tfidf
```

```
tfidfA = computeTFIDF(tfA, idfs)
tfidfB = computeTFIDF(tfB, idfs)
df = pd.DataFrame([tfidfA, tfidfB])
df
```

	largest	Planet	planet	is	Mars	from	Jupyter	Sun	the	fourth
0	0.138629	0.138629	0.000000	0.0	0.000000	0.000000	0.138629	0.000000	0.0	0.000000
1	0.000000	0.000000	0.099021	0.0	0.099021	0.099021	0.000000	0.099021	0.0	0.099021

Next steps: Generate code with df | New interactive sheet